

Summary

Tail call optimization is a feature of a programming language's compiler or interpreter that can be employed on recursive functions specifically written to be tail recursive. Tail recursive functions return the return value of the recursive function call as the last action in the recursive case. This allows the function to delete the current frame object and prevent the call stack from growing as new recursive function calls are made. If the call stack doesn't grow, the recursive function can't possibly cause a stack overflow.

Tail recursion is a workaround that allows some recursive algorithms to work with large arguments without crashing. However, this approach requires rearranging your code and possibly adding an accumulator parameter. This could make your code harder to understand. You may likely find that sacrificing code readability is not worth using a recursive algorithm over an iterative one.